



22MAT230 MATHEMATICS FOR COMPUTING 4

Batch : AIE -C

Team C-11

- | | |
|--------------------|--------------------|
| • Jovika N B | - CB.SC.U4AIE24223 |
| • P Poojitha | - CB.SC.U4AIE24237 |
| • Ramu Jahna Bindu | - CB.SC.U4AIE24249 |
| • Potnuru Varun | - CB.SC.U4AIE24244 |

Density-Adaptive Graph Laplacian Construction for Robust Spectral Clustering

Deep Feature Based Land-Use Image Clustering using ResNet Embeddings

1.Introduction :

Remote sensing imagery plays a crucial role in many modern applications such as **urban planning, environmental monitoring, land-use analysis, and disaster management**. Satellite images contain large amounts of visual information that must be organized and interpreted efficiently. However, manual analysis of satellite images is time-consuming and prone to human error. Therefore, **automatic image understanding using machine learning techniques has become an important research area**.

Traditional image clustering approaches often rely on **hand-crafted features** such as color histograms, textures, or edge descriptors. Although these features can capture certain visual properties, they are often insufficient for representing complex spatial patterns present in satellite images. Recently, **deep learning models trained on large datasets have demonstrated superior ability to learn high-level semantic features** from images. In particular, convolutional neural networks (CNNs) such as **ResNet** can extract meaningful representations that capture structures like vegetation patterns, road networks, water bodies, and urban layouts.

Despite the success of deep neural networks, clustering high-dimensional image features remains a challenging task. Classical clustering algorithms such as **K-Means** assume that clusters are spherical and separable in Euclidean space. However, real-world image data often lies on complex nonlinear manifolds where such

assumptions do not hold. To address this limitation, **spectral clustering** has emerged as a powerful technique that models the dataset as a graph and exploits the intrinsic geometric structure of the data.

In this project, we propose a **deep feature based clustering framework for satellite image analysis**. First, deep features are extracted from satellite images using a pretrained **ResNet-50** network. These high-level representations capture semantic information about the scene. The extracted features are then clustered using three different approaches:

- **K-Means clustering**
- **Spectral clustering with global Gaussian kernel**
- **Self-tuning spectral clustering with adaptive local scaling**

The goal of this work is to evaluate how different clustering strategies perform when applied to deep feature representations of satellite images. The performance of the clustering methods is evaluated using metrics such as **Clustering Accuracy (ACC)**, **Normalized Mutual Information (NMI)**, and **Adjusted Rand Index (ARI)**.

The experiments are conducted on the **EuroSAT dataset**, which contains satellite images from multiple land-use categories including highway, residential, forest, river, and sea/lake regions. Experimental results demonstrate that **self-tuning spectral clustering provides the best performance**, highlighting the importance of adaptive similarity measures for high-dimensional image embeddings.

This project demonstrates how combining **deep learning feature extraction with advanced clustering techniques** can significantly improve the organization and understanding of remote sensing imagery.

2. Problem Statement :

Satellite imagery contains large volumes of visual information that describe land-use patterns such as forests, highways, residential regions, rivers, and water bodies. Efficiently organizing these images is essential for applications including environmental monitoring, urban planning, and geographical analysis. However, manually categorizing large-scale satellite image datasets is time-consuming and impractical.

Traditional clustering approaches rely on handcrafted features such as color, texture, and shape descriptors. These features often fail to capture the complex semantic patterns present in satellite imagery. As a result, clustering performance becomes limited when dealing with high variability in real-world scenes.

Although deep convolutional neural networks can extract powerful representations from images, clustering these **high-dimensional deep feature vectors** remains challenging. Many conventional clustering algorithms, such as K-Means, assume that clusters are linearly separable and spherical in structure, which may not reflect the true distribution of image features.

Therefore, the main challenge addressed in this project is to develop a framework that can **effectively cluster satellite images using deep feature representations while preserving the intrinsic structure of the data**.

To address this problem, deep features extracted from pretrained neural networks are combined with graph-based clustering techniques such as spectral clustering and self-tuning spectral clustering.

3. Objectives of the Project

The primary objectives of this project are:

1. To extract meaningful deep feature representations from satellite images using pretrained convolutional neural networks.
2. To apply clustering algorithms to group satellite images into meaningful land-use categories.
3. To compare different clustering approaches including K-Means clustering and spectral clustering methods.
4. To analyze the effectiveness of self-tuning spectral clustering for handling high-dimensional image embeddings.
5. To evaluate clustering performance using metrics such as **Clustering Accuracy (ACC)**, **Normalized Mutual Information (NMI)**, and **Adjusted Rand Index (ARI)**.
6. To visualize high-dimensional feature representations using **Principal Component Analysis (PCA)** for better understanding of cluster separation.

4. Dataset Description

Dataset Name :

The dataset used in this project is the EuroSAT Dataset.

EuroSAT is a satellite image dataset for land-use and land-cover classification derived from Sentinel-2 satellite imagery

The EuroSAT dataset contains satellite images of different land-use categories.

- Original dataset characteristics:
- Total images: 27,000
- Number of classes: 10
- Image size: 64 × 64 pixels
- Image type: RGB satellite images
- Each image represents a specific land-use class.

We selected five classes from the dataset:

- Highway
- Residential
- Forest
- River
- SeaLake

These classes were selected because they represent distinct land-use patterns and provide a good clustering challenge.

Dataset Size Used :

To maintain a balanced dataset, we selected:

- 400 images per class

Therefore,

- Resolution: 224×224 pixels (we scale it up)
- Total Images= $5 \times 400 = 2000$
- 2000 satellite images

5 land-use classes (balanced distribution)

Balanced datasets help prevent bias in clustering algorithms.

Image Properties :

Each image has the following characteristics:

Resolution: 64×64 pixels (we scale it upto 224×224 pixels)

Color channels: RGB (3 channels)

Image format: JPEG

Data type: satellite imagery

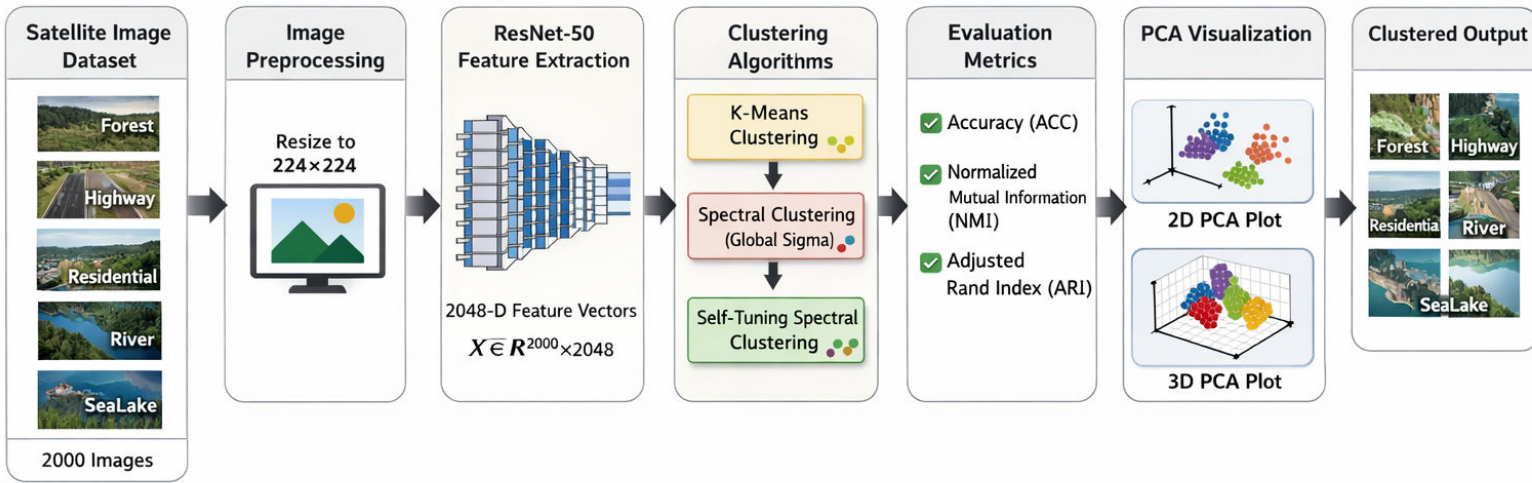
These images represent top-down views of Earth's surface captured by satellites.

5. Methodology

The proposed framework consists of several stages:

1. Dataset Loading
2. Deep Feature Extraction
3. Clustering Algorithms
4. Evaluation Metrics
5. Visualization using PCA

Project Architecture for Deep Feature Extraction and Self-Tuning Spectral Clustering



5.1 Dataset Loading :

The dataset used in this project is the **EuroSAT satellite image dataset**, which contains images representing different land-use categories. From the complete dataset, five classes were selected for the experiments:

- Highway
- Residential
- Forest
- SeaLake
- River

To maintain a balanced dataset, **400 images were selected from each class**, resulting in a total of :
2000 images

The dataset was loaded in MATLAB using the `imageDatastore` function. This function reads images from the dataset directory and automatically assigns labels based on the folder names in which the images are stored. This makes it easier to manage large collections of images.

After loading the dataset, only the required classes were selected and the labels were cleaned to remove unused categories. The dataset was then balanced by randomly selecting **400 images from each class** using MATLAB's `splitEachLabel` function.

Finally, the categorical class labels were converted into numeric labels so that they could be used for clustering evaluation metrics later in the experiment.

After the dataset loading and preprocessing step, the final dataset contained **2000 labeled satellite images**, which were then used for deep feature extraction and clustering analysis.

5.2 Deep Feature Extraction :

After loading and preprocessing the dataset, the next step is to extract meaningful features from the satellite images. Raw images contain thousands of pixel values, which are not always suitable for clustering because they include noise and redundant information. Therefore, instead of using raw pixels, **deep feature representations** are extracted using a pretrained deep learning model.

Deep feature extraction allows the model to represent images using **high-level semantic information**, such as textures, shapes, and structural patterns present in satellite imagery.

- Deep feature extraction is the process of using a **deep learning model** to automatically learn meaningful representations from images. Instead of using raw pixel values, the neural network transforms each image into a **feature vector** that captures important visual characteristics.
- In traditional image processing, features such as edges, color histograms, and textures are manually designed by humans. However, these handcrafted features often fail to capture complex patterns in images. Deep learning models solve this problem by **learning features automatically from large datasets**.
- In this project, deep feature extraction is performed using a **pretrained ResNet-50 convolutional neural network**.

Why Feature Extraction is Needed ?

Images are composed of thousands of pixel values. For example, an RGB image of size: $224 \times 224 \times 3$ contains more than **150,528 pixel values**.

Using raw pixels for clustering is not effective because:

- Pixel values contain noise
- Images may vary in brightness or orientation
- Raw pixels do not represent meaningful patterns

Therefore, images are converted into **compact feature vectors** that describe important properties of the image.

Deep Neural Networks for Feature Extraction :

Convolutional Neural Networks (CNNs) learn features in a **hierarchical manner**.

The layers of the network learn different levels of visual information:

Early Layers

Detect simple patterns such as

- edges
- lines
- corners

Middle Layers

Detect more complex patterns such as

- textures
- repeated shapes
- local structures

Deep Layers

Detect high-level semantic patterns such as

- roads
- buildings
- water bodies
- vegetation areas

Because satellite images contain complex spatial patterns, deep neural networks are very effective for extracting meaningful representations.

Using ResNet-50 for Feature Extraction :

Deep feature extraction is performed using a pretrained **ResNet-50 convolutional neural network**. ResNet-50 is a deep learning architecture designed for image analysis and was trained on the **ImageNet dataset**, which contains millions of labeled images across thousands of categories. Because of this large-scale training, the network has learned rich visual representations that can be transferred to other image analysis tasks such as clustering and similarity detection.

Instead of using the network for classification, ResNet-50 is used as a **feature extractor**. This means that images are passed through the network, and the internal feature representations learned by the network are extracted and used as numerical descriptors of the images.

Structure of ResNet-50 :

ResNet-50 is composed of **50 layers**, including convolution layers, activation functions, pooling layers, and residual blocks. One of the key innovations of ResNet is the use of **residual connections**, which allow the network to learn deeper representations without suffering from the vanishing gradient problem.

The residual learning mechanism can be expressed as:

$$H(x) = F(x) + x$$

where:

- x represents the input to a layer
- $F(x)$ represents the learned residual mapping
- $H(x)$ represents the output of the residual block
- These skip connections help the network preserve information while learning complex transformations.
- Because of this architecture, ResNet-50 is capable of learning highly discriminative visual feature

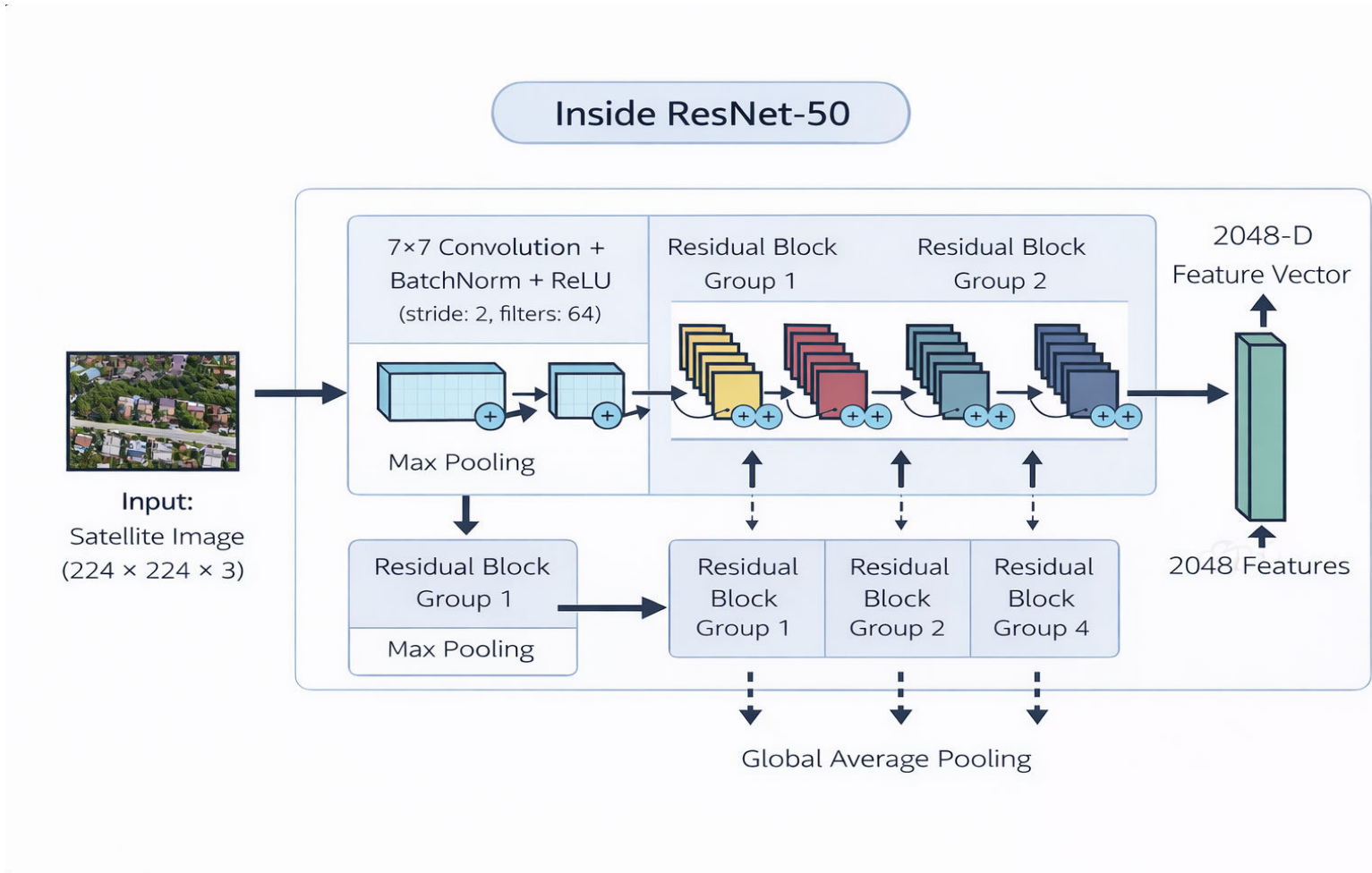


FIG : Resnet 50 architecture diagram

Image Preprocessing :

Before images are processed by ResNet-50, they must be resized to match the network's input requirements. The network expects input images of size: 224×224×3

where the three channels represent the RGB color components.

In MATLAB, this resizing is performed using **augmentedImageDatastore**, which automatically adjusts the image dimensions and feeds the images into the network in batches.

This preprocessing step ensures that all images have a consistent format before feature extraction.

Feature Extraction Layer :

Instead of using the final classification layer of the network, features are extracted from the **global average pooling layer**, known as the **avg_pool layer**.

This layer is located near the end of the network and produces a compact representation of the learned feature maps.

The global average pooling operation computes the average value of each feature map across its spatial dimensions. If a feature map has dimensions $H \times W$ the pooled feature value is computed as :

$$f_k = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W a_{ij}^{(k)}$$

where:

- $a_{ij}^{(k)}$ represents the activation value at location (i, j) in feature map k
- H and W represent the spatial dimensions
- f_k represents the pooled output value

ResNet-50 produces **2048 feature maps** in the final convolutional stage. After global average pooling, each feature map is summarized into a single value. Therefore, the final feature vector contains : 2048

features for each image.

Feature Representation of the Dataset :

After passing each image through the ResNet-50 network, the output of the global average pooling layer is extracted as a **feature vector**.

If the dataset contains:

$N=2000$

images and each image produces a **2048-dimensional feature vector**, the final feature matrix becomes:

$$X \in \mathbb{R}^{2000 \times 2048}$$

where:

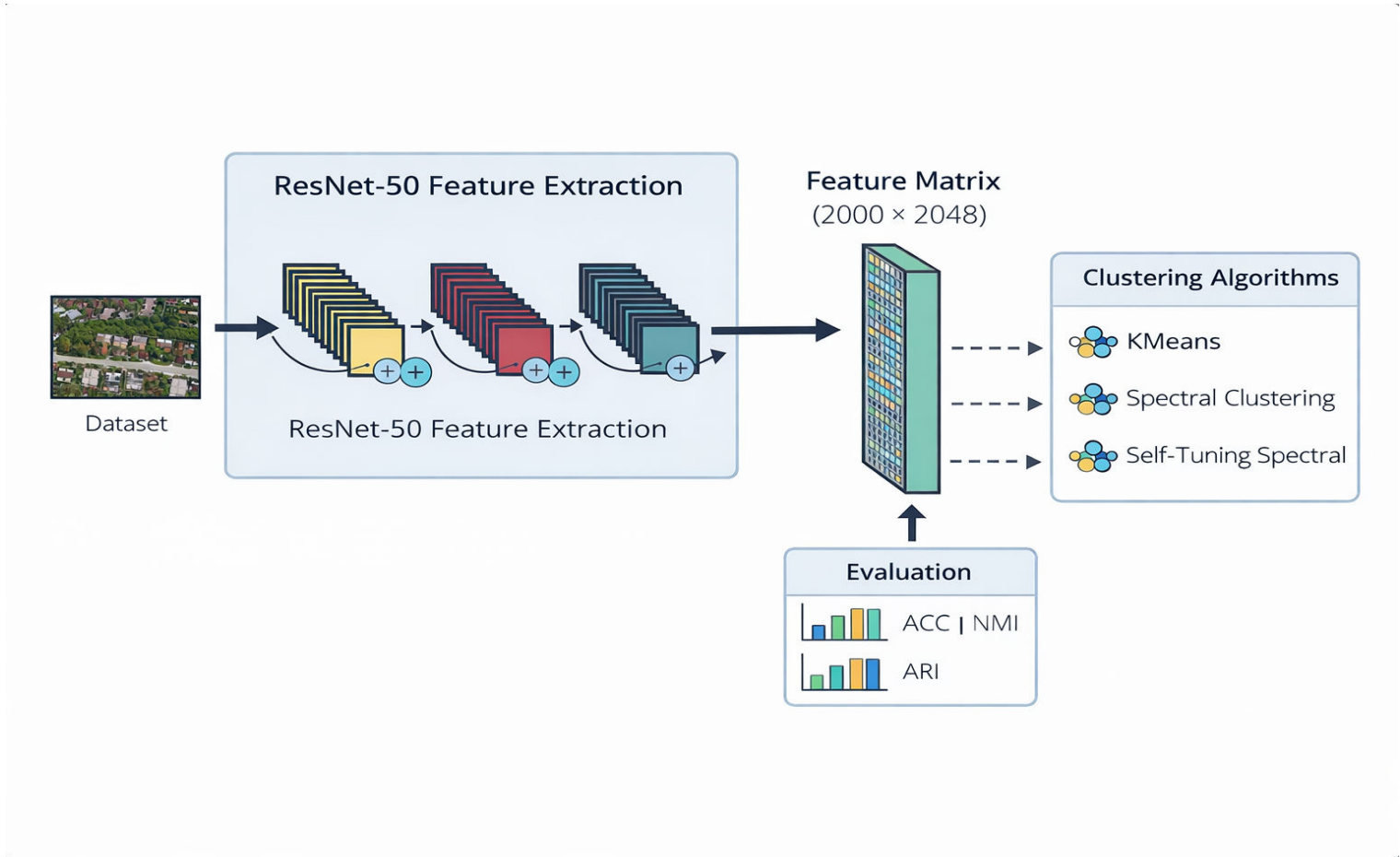
- each **row** represents one satellite image
- each **column** represents a learned feature dimension

This matrix serves as the input to the clustering algorithms used in the project.

The features extracted by ResNet-50 capture high-level semantic information about the satellite images. These features encode complex visual patterns such as:

- vegetation textures in forest regions
- linear road structures in highways
- building layouts in residential areas
- water patterns in rivers and lakes

Because these features represent meaningful visual structures rather than raw pixel values, they allow clustering algorithms to group similar images more accurately.



6. Clustering Algorithms :

After extracting deep features using the **ResNet-50 network**, each satellite image is represented by a **2048-dimensional feature vector**. These vectors form a feature matrix :

$$X \in \mathbb{R}^{N \times d}$$

where

- N=2000(number of images)
- d=2048 (feature dimension)

The goal of clustering is to **group similar images together without using label information**. In this project, three clustering algorithms are used:

1. **K-Means Clustering**
2. **Spectral Clustering (Global Sigma)**
3. **Self-Tuning Spectral Clustering**

These algorithms analyze the deep feature representations to identify natural groupings within the dataset.

6.1 : K-Means Clustering :

K-Means is one of the most commonly used unsupervised clustering algorithms. It partitions the dataset into K clusters by minimizing the distance between data points and the centroid of their assigned cluster.

The objective of K-Means is to minimize the **within-cluster variance**, defined as:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

where

- C_k = cluster k
- μ_k = centroid of cluster k
- x_i = feature vector of image iii

Working Steps of K-Means

- **Initialization :**

Randomly initialize K centroids.

$$\mu_1, \mu_2, \dots, \mu_K$$

- **Cluster Assignment :**

Each data point is assigned to the nearest centroid using Euclidean distance:

$$C_k = \{x_i : ||x_i - \mu_k|| \leq ||x_i - \mu_j||\}$$

- **Centroid Update :**

The centroid of each cluster is recomputed as:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k}$$

- **Iteration :**

Steps 2 and 3 are repeated until cluster assignments stabilize.

Application in This Project :

The number of clusters is chosen as : K=5

because the dataset contains **five land-use classes**:

- Highway
- Residential
- Forest
- River
- SeaLake

K-Means provides a baseline clustering performance for the extracted deep features.

6.2 : Spectral Clustering (Global Sigma)

K-Means assumes clusters are **spherical in Euclidean space**, which may not hold for complex image datasets. Spectral clustering addresses this limitation by modeling the dataset as a **graph**.

Each image is treated as a node in the graph, and edges represent similarity between images.

Distance Matrix :

First, pairwise distances between all feature vectors are computed :

$$D_{ij} = ||x_i - x_j||_2$$

This results in a **distance matrix**:

$$D \in \mathbb{R}^{N \times N}$$

Properties of this matrix:

- $D_{ii} = 0$
- symmetric matrix

Gaussian Similarity Function :

To convert distances into similarities, a Gaussian kernel is used:

$$W_{ij} = \exp \left(-\frac{||x_i - x_j||^2}{2\sigma^2} \right)$$

where ;

σ is a scale parameter controlling similarity decay.

In this project :

$$\sigma = \text{median}(D)$$

- Using the median provides a **robust estimate of the dataset scale**.
- The resulting matrix W is called the **affinity matrix**, representing the similarity between all image pairs.

Graph Laplacian :

Next, the **degree matrix** is computed:

$$D_g = \text{diag}\left(\sum_j W_{ij}\right)$$

The normalized graph Laplacian is defined as:

$$L = I - D_g^{-1/2} W D_g^{-1/2}$$

where

- W = affinity matrix
- D_g = degree matrix
- I = identity matrix

The Laplacian captures the **structure of the similarity graph**.

Spectral Embedding :

Eigen decomposition is performed:

$$L u_i = \lambda_i u_i$$

The eigenvectors corresponding to the **smallest eigenvalues** are selected:

$$U = [u_1, u_2, \dots, u_K]$$

where ,

$$U \in \mathbb{R}^{N \times K}$$

- Each row of U represents a new low-dimensional embedding of the data.
- Finally, **K-Means clustering is applied to this embedding space** to obtain the final clusters.

6.3 : Self-Tuning Spectral Clustering

Global sigma assumes a **single similarity scale for the entire dataset**, which may not work well when data density varies across regions.

Self-tuning spectral clustering improves this by using **local scaling parameters**.

1. Local Scale Parameter :

For each data point x_i , a local scale parameter is computed as:

$$\sigma_i = d(x_i, x_k)$$

where

x_k **is the k-th nearest neighbor of x_i .**

In this project:

$$k = 25$$

This means the distance to the **25th nearest neighbor** defines the local scale.

Why Use the 25th Nearest Neighbor?

This helps measure **local density**.

Dense Region :

Points are close together.

• • • •
• • • •

Distance to the 25th neighbor is **small**.

So:

σ_i is small

This means similarity drops quickly.

Sparse Region :

Points are far apart.

• •
•

Distance to the 25th neighbor is **large**.

So:

σ_i is large σ_i is large

Similarity decreases slowly.

2. Adaptive Similarity Function :

Similarity between two points becomes:

$$W_{ij} = \exp \left(- \frac{\|x_i - x_j\|^2}{\sigma_i \sigma_j} \right)$$

This adaptive kernel allows the similarity measure to **adjust based on local neighborhood density**.

7. Evaluation Metrics

To evaluate the performance of the clustering algorithms, several quantitative metrics are used. These metrics measure how well the predicted clusters match the true class labels of the dataset.

In this project, three evaluation metrics are used:

- **Clustering Accuracy (ACC)**
- **Normalized Mutual Information (NMI)**
- **Adjusted Rand Index (ARI)**

These metrics provide a comprehensive evaluation of clustering quality.

7.1 Clustering Accuracy (ACC)

Clustering Accuracy measures the proportion of correctly clustered samples after matching cluster labels with true labels.

Since clustering algorithms assign arbitrary cluster indices, a mapping is required between predicted clusters and ground truth labels.

The clustering accuracy is computed as:

$$ACC = \frac{1}{N} \sum_{i=1}^N \delta(y_i, \text{map}(c_i))$$

where

- N = total number of samples
- y_i = true label of sample i
- c_i = predicted cluster label
- $\text{map}(\cdot)$ = mapping function aligning clusters with labels
- δ = indicator function

The indicator function is defined as :

$$\delta(a, b) = \begin{cases} 1 & a = b \\ 0 & a \neq b \end{cases}$$

A higher ACC value indicates better clustering performance.

7.2 Normalized Mutual Information (NMI)

Normalized Mutual Information measures the similarity between the true labels and the predicted clusters using information theory.

It evaluates how much information about the true labels is captured by the clustering result.

The mutual information is defined as :

$$MI(Y, C) = \sum_i \sum_j P(i, j) \log \frac{P(i, j)}{P(i)P(j)}$$

where

- Y represents true labels
- C represents cluster labels
- $P(i, j)$ represents the joint probability

NMI is computed as:

$$NMI = \frac{MI(Y, C)}{\sqrt{H(Y)H(C)}}$$

where

- $H(Y)$ = entropy of true labels
- $H(C)$ = entropy of cluster labels

The NMI value ranges between **0** and **1**.

- **0** → no similarity
- **1** → perfect clustering

7.3 Adjusted Rand Index (ARI)

The Adjusted Rand Index measures the similarity between two clustering assignments by comparing pairwise relationships between samples.

It evaluates whether pairs of points are assigned to the same cluster in both predicted and true labels.

ARI is defined as :

$$ARI = \frac{RI - Expected}{Max - Expected}$$

where

- RI = Rand Index
- ExpectedExpectedExpected = expected similarity of random clustering

ARI ranges from **-1 to 1**.

- **1** → perfect clustering
- **0** → random clustering
- **negative values** → poor clustering

8 .PCA Visualization of Clusters :

Principal Component Analysis (PCA) is used to visualize the distribution of deep feature representations extracted from the ResNet-50 network. Since the extracted feature vectors have a high dimensionality of **2048**, it is difficult to directly visualize the clustering structure in such a high-dimensional space. PCA helps overcome this limitation by projecting the data into a lower-dimensional space while preserving the most important variance in the dataset.

In this project, PCA is applied to the deep feature matrix:

$$X \in \mathbb{R}^{2000 \times 2048}$$

where each row represents the feature vector of a satellite image.

PCA transforms this high-dimensional feature space into a new set of orthogonal axes called **principal components**. These components capture the directions of maximum variance in the data.

2D PCA Visualization

For two-dimensional visualization, the first two principal components are selected :

$$Z_{2D} = [PC_1, PC_2]$$

These components represent the directions that preserve the largest amount of variance in the dataset. The feature vectors are projected onto this two-dimensional space and plotted using scatter plots.

The 2D visualization is used to display:

- True class labels
- K-Means clustering results

- Spectral clustering results
- Self-tuning spectral clustering results

By comparing these plots, it becomes possible to observe how well each clustering algorithm separates different satellite image categories.

3D PCA Visualization

To gain a better understanding of the data structure, PCA is also used to generate three-dimensional visualizations. In this case, the first three principal components are selected:

$$Z_{3D} = [PC_1, PC_2, PC_3]$$

These three components capture a larger portion of the dataset variance and allow better separation of clusters in the visualization space.

Each satellite image is plotted as a point in the 3D PCA space, where:

- each axis represents a principal component
- each color represents a cluster or class

This visualization helps identify patterns such as:

- cluster separation
- cluster overlap
- distribution of samples across clusters

Interpretation of PCA Plots

The PCA visualizations provide an intuitive understanding of how well the clustering algorithms group similar satellite images.

If clusters appear well separated in the PCA plots, it indicates that the deep features extracted by ResNet-50 effectively capture meaningful visual differences between land-use categories.

In this project, the PCA plots demonstrate that **self-tuning spectral clustering produces better separated clusters compared to K-Means and global spectral clustering**, confirming the improved clustering performance observed in the evaluation metrics.